1. REACT IMPLEMENTATION CODE

```
import React, { useState } from "react";
import "./FramePlanComponents.css";

// Shared status component
function StatusMessage({ status, message }) {
  if (!status || status === "default") return null;

  const messages = {
    loading: "Loading information...",
    empty: message || "No information available yet.",
    error: message || "Something went wrong. Please try again.",
    success: message || "Changes saved successfully.",
    disabled: "This action is currently unavailable.",
    validation: message || "Please complete the required fields."
  };

  return (
    <div className={`status-message ${status}`} role="status">
      {messages[status]}
    </div>
  );
}

// ========================================
// COMPONENT 1: PROJECT HUB
// ========================================

export function ProjectHub({
  initialProjects = [],
  status = "default",
  onCreateProject,
  onOpenProject,
  onEditProject,
  onDeleteProject
}) {
  const [projects, setProjects] = useState(initialProjects);
  const [newTitle, setNewTitle] = useState("");
  const [newDescription, setNewDescription] = useState("");
  const [showForm, setShowForm] = useState(false);
  const [error, setError] = useState("");

  function addProject(e) {
    e.preventDefault();

    if (!newTitle.trim()) {
```

```
    setError("Please enter a project title.");
    return;
  }

  const project = {
    id: Date.now(),
    title: newTitle,
    description: newDescription,
    scenes: 0,
    shots: 0,
    progress: 0,
    type: "Short Film"
  };

  setProjects(previous => [...previous, project]);
  onCreateProject?.(project);

  setNewTitle("");
  setNewDescription("");
  setError("");
  setShowForm(false);
}

function deleteProject(id) {
  if (!window.confirm("Delete this project?")) return;

  setProjects(previous =>
    previous.filter(project => project.id !== id)
  );

  onDeleteProject?.(id);
}

return (
  <section className="frame-section">
    <div className="section-heading">
      <div>
        <span className="eyebrow">FRAMEPLAN</span>
        <h2>Project Hub</h2>
        <p>Create and manage your filmmaking projects.</p>
      </div>

      <button
        className="primary-button"
        onClick={() => setShowForm(!showForm)}
      >
        + Create New Project
      </button>
```

```
</div>

{showForm && (
  <form className="frame-card form-card" onSubmit={addProject}>
    <h3>Create a Project</h3>

    <label htmlFor="project-title">Project Title</label>
    <input
      id="project-title"
      value={newTitle}
      onChange={e => setNewTitle(e.target.value)}
      placeholder="Enter project title"
      required
    />

    <label htmlFor="project-description">Description</label>
    <textarea
      id="project-description"
      value={newDescription}
      onChange={e => setNewDescription(e.target.value)}
      placeholder="Describe your film project"
    />

    {error && <p className="error-text">{error}</p>}

    <button className="primary-button" type="submit">
      Save Project
    </button>
    <button
      type="button"
      className="secondary-button"
      onClick={() => setShowForm(false)}
    >
      Cancel
    </button>
  </form>
)}

<StatusMessage status={status} />

{status === "default" && projects.length === 0 ? (
  <StatusMessage
    status="empty"
    message="No projects yet. Create your first filmmaking project!"
  />
) : (
  <div className="project-grid">
    {projects.map(project => (
```

```
      <article className="frame-card project-card" key={project.id}>
        <span className="project-label">
          {project.type || "Film Project"}
        </span>

        <h3>{project.title}</h3>
        <p>{project.description}</p>

        <div className="project-stats">
          <span>Scenes: {project.scenes}</span>
          <span>Shots: {project.shots}</span>
        </div>

        <div className="progress-track">
          <div
            className="progress-fill"
            style={{ width: `${project.progress || 0}%` }}
          />
        </div>

        <p>{project.progress || 0}% planned</p>

        <div className="card-actions">
          <button
            className="primary-button"
            onClick={() => onOpenProject?.(project)}
          >
            Continue Planning
          </button>

          <button
            className="secondary-button"
            onClick={() => onEditProject?.(project)}
          >
            Edit
          </button>

          <button
            className="danger-button"
            onClick={() => deleteProject(project.id)}
          >
            Delete
          </button>
        </div>
      </article>
    ))}
  </div>
)}
```

```
    </section>
  );
}

// ========================================
// COMPONENT 2: SCENE BUILDER
// ========================================

export function SceneBuilder({
  initialScenes = [],
  status = "default",
  onAddScene,
  onEditScene,
  onDeleteScene,
  onArrangeScenes
}) {
  const [scenes, setScenes] = useState(initialScenes);
  const [title, setTitle] = useState("");
  const [location, setLocation] = useState("");
  const [description, setDescription] = useState("");
  const [showForm, setShowForm] = useState(false);
  const [customBoxes, setCustomBoxes] = useState([]);
  const [customName, setCustomName] = useState("");
  const [teamMembers, setTeamMembers] = useState([]);
  const [memberName, setMemberName] = useState("");

  function addScene(e) {
    e.preventDefault();

    if (!title.trim()) return;

    const scene = {
      id: Date.now(),
      number: scenes.length + 1,
      title,
      location,
      description,
      shots: []
    };

    setScenes(previous => [...previous, scene]);
    onAddScene?.(scene);

    setTitle("");
    setLocation("");
    setDescription("");
    setShowForm(false);
  }
```

```
function deleteScene(id) {
  if (!window.confirm("Delete this scene?")) return;

  setScenes(previous =>
    previous
      .filter(scene => scene.id !== id)
      .map((scene, index) => ({
        ...scene,
        number: index + 1
      }))
  );

  onDeleteScene?.(id);
}

function moveScene(index, direction) {
  const next = [...scenes];
  const target = index + direction;

  if (target < 0 || target >= next.length) return;

  [next[index], next[target]] = [next[target], next[index]];

  const reordered = next.map((scene, i) => ({
    ...scene,
    number: i + 1
  }));

  setScenes(reordered);
  onArrangeScenes?.(reordered);
}

function addCustomBox(e) {
  e.preventDefault();
  if (!customName.trim()) return;

  setCustomBoxes(previous => [...previous, customName]);
  setCustomName("");
}

function addTeamMember(e) {
  e.preventDefault();
  if (!memberName.trim()) return;

  setTeamMembers(previous => [...previous, memberName]);
  setMemberName("");
}
```

```
return (
  <section className="frame-section">
    <div className="section-heading">
      <div>
        <span className="eyebrow">PROJECT PLANNING</span>
        <h2>Scene Builder</h2>
        <p>Organise your scenes and plan your shooting order.</p>
      </div>

      <button
        className="primary-button"
        onClick={() => setShowForm(!showForm)}
      >
        + Add Scene
      </button>
    </div>

    <StatusMessage status={status} />

    {showForm && (
      <form className="frame-card form-card" onSubmit={addScene}>
        <h3>New Scene</h3>

        <label htmlFor="scene-title">Scene Title</label>
        <input
          id="scene-title"
          value={title}
          onChange={e => setTitle(e.target.value)}
          placeholder="Enter scene title"
          required
        />

        <label htmlFor="scene-location">Location</label>
        <input
          id="scene-location"
          value={location}
          onChange={e => setLocation(e.target.value)}
          placeholder="Where does this scene take place?"
        />

        <label htmlFor="scene-description">Description</label>
        <textarea
          id="scene-description"
          value={description}
          onChange={e => setDescription(e.target.value)}
          placeholder="Describe what happens in this scene"
        />
```

```
    <button className="primary-button" type="submit">
      Save Scene
    </button>
    <button
      type="button"
      className="secondary-button"
      onClick={() => setShowForm(false)}
    >
      Cancel
    </button>
  </form>
)}

{scenes.length === 0 ? (
  <StatusMessage
    status="empty"
    message="No scenes yet. Add your first scene to start planning."
  />
) : (
  <div className="scene-list">
    {scenes.map((scene, index) => (
      <article className="frame-card scene-card" key={scene.id}>
        <span className="project-label">
          Scene {scene.number}
        </span>

        <h3>{scene.title}</h3>
        <p>{scene.description}</p>
        <p>Location: {scene.location || "Not specified"}</p>
        <p>Shots: {scene.shots?.length || 0}</p>

        <div className="card-actions">
          <button
            className="secondary-button"
            onClick={() => onEditScene?.(scene)}
          >
            Edit
          </button>

          <button
            className="secondary-button"
            onClick={() => moveScene(index, -1)}
            disabled={index === 0}
          >
            Move Up
          </button>
```

```
          <button
            className="secondary-button"
            onClick={() => moveScene(index, 1)}
            disabled={index === scenes.length - 1}
          >
            Move Down
          </button>

          <button
            className="danger-button"
            onClick={() => deleteScene(scene.id)}
          >
            Delete
          </button>
        </div>
      </article>
    ))}
  </div>
)}

<div className="extra-sections">
  <div className="frame-card">
    <h3>Custom Boxes</h3>
    <form onSubmit={addCustomBox}>
      <label htmlFor="custom-box">Create a custom section</label>
      <input
        id="custom-box"
        value={customName}
        onChange={e => setCustomName(e.target.value)}
        placeholder="Enter section name"
      />
      <button className="primary-button" type="submit">
        Add Box
      </button>
    </form>

    {customBoxes.map((box, index) => (
      <p key={index}>{box}</p>
    ))}
  </div>

  <div className="frame-card">
    <h3>Team Members</h3>
    <form onSubmit={addTeamMember}>
      <label htmlFor="team-member">Add a team member</label>
      <input
        id="team-member"
        value={memberName}
```

```
            onChange={e => setMemberName(e.target.value)}
            placeholder="Enter member name"
           />
           <button className="primary-button" type="submit">
            Add Member
           </button>
          </form>

          {teamMembers.map((member, index) => (
           <p key={index}>{member}</p>
          ))}
        </div>
      </div>
    </section>
  );
}

// ========================================
// COMPONENT 3: SHOT STUDIO
// ========================================

export function ShotStudio({
  initialShots = [],
  status = "default",
  onAddShot,
  onEditShot,
  onDeleteShot,
  onShare
}) {
  const [shots, setShots] = useState(initialShots);
  const [view, setView] = useState("table");
  const [showForm, setShowForm] = useState(false);

  const [form, setForm] = useState({
    title: "",
    description: "",
    size: "",
    angle: "",
    movement: "",
    notes: ""
  });

  const [error, setError] = useState("");

  function updateField(e) {
    setForm(previous => ({
      ...previous,
      [e.target.name]: e.target.value
```

```
  }));
}

function addShot(e) {
  e.preventDefault();

  if (!form.title.trim()) {
    setError("Please enter a shot title.");
    return;
  }

  const shot = {
    id: Date.now(),
    number: shots.length + 1,
    ...form
  };

  setShots(previous => [...previous, shot]);
  onAddShot?.(shot);

  setForm({
    title: "",
    description: "",
    size: "",
    angle: "",
    movement: "",
    notes: ""
  });

  setError("");
  setShowForm(false);
}

function deleteShot(id) {
  if (!window.confirm("Delete this shot?")) return;

  setShots(previous =>
    previous
      .filter(shot => shot.id !== id)
      .map((shot, index) => ({
        ...shot,
        number: index + 1
      }))
  );

  onDeleteShot?.(id);
}
```

```
function moveShot(index, direction) {
  const next = [...shots];
  const target = index + direction;

  if (target < 0 || target >= next.length) return;

  [next[index], next[target]] = [next[target], next[index]];

  const reordered = next.map((shot, i) => ({
    ...shot,
    number: i + 1
  }));

  setShots(reordered);
}

function printShots() {
  window.print();
}

function shareShots() {
  const text = shots.map(shot =>
    `Shot ${shot.number}: ${shot.title}\n` +
    `Size: ${shot.size}\n` +
    `Angle: ${shot.angle}\n` +
    `Movement: ${shot.movement}\n` +
    `Notes: ${shot.notes}`
  ).join("\n\n");

  if (navigator.share) {
    navigator.share({
      title: "FramePlan Shot List",
      text
    }).catch(() => {});
  } else if (navigator.clipboard) {
    navigator.clipboard.writeText(text);
    onShare?.(text);
    alert("Shot list copied. You can now share it.");
  } else {
    onShare?.(text);
  }
}

return (
  <section className="frame-section">
    <div className="section-heading">
      <div>
        <span className="eyebrow">FILM PLANNING</span>
```

```
    <h2>Shot Studio</h2>
    <p>Preview, edit, and organise every shot.</p>
   </div>

   <div className="card-actions">
    <button className="primary-button" onClick={shareShots}>
     Share
    </button>
    <button className="secondary-button" onClick={printShots}>
     Print
    </button>
   </div>
  </div>

  <div className="toolbar">
   <button
    className={view === "table" ? "primary-button" : "secondary-button"}
    onClick={() => setView("table")}
    aria-pressed={view === "table"}
   >
    Table
   </button>

   <button
    className={view === "cards" ? "primary-button" : "secondary-button"}
    onClick={() => setView("cards")}
    aria-pressed={view === "cards"}
   >
    Cards
   </button>

   <button
    className="primary-button"
    onClick={() => setShowForm(!showForm)}
   >
    + Add Shot
   </button>
  </div>

  <StatusMessage status={status} />

  {showForm && (
   <form className="frame-card form-card" onSubmit={addShot}>
    <h3>Add a New Shot</h3>

    <label htmlFor="shot-title">Shot Title *</label>
    <input
     id="shot-title"
```

```
  name="title"
  value={form.title}
  onChange={updateField}
  placeholder="Enter shot title"
  required
/>

<label htmlFor="shot-description">Description</label>
<textarea
  id="shot-description"
  name="description"
  value={form.description}
  onChange={updateField}
  placeholder="Describe the shot"
/>

<label htmlFor="shot-size">Shot Size</label>
<select
  id="shot-size"
  name="size"
  value={form.size}
  onChange={updateField}
>
  <option value="">Select shot size</option>
  <option>Extreme Wide Shot</option>
  <option>Wide Shot</option>
  <option>Full Shot</option>
  <option>Medium Shot</option>
  <option>Medium Close-Up</option>
  <option>Close-Up</option>
  <option>Extreme Close-Up</option>
</select>

<label htmlFor="shot-angle">Camera Angle</label>
<select
  id="shot-angle"
  name="angle"
  value={form.angle}
  onChange={updateField}
>
  <option value="">Select camera angle</option>
  <option>Eye Level</option>
  <option>High Angle</option>
  <option>Low Angle</option>
  <option>Bird's-Eye View</option>
  <option>Worm's-Eye View</option>
  <option>Dutch Angle</option>
</select>
```

```
    <label htmlFor="shot-movement">Camera Movement</label>
    <select
      id="shot-movement"
      name="movement"
      value={form.movement}
      onChange={updateField}
    >
      <option value="">Select camera movement</option>
      <option>Static</option>
      <option>Pan</option>
      <option>Tilt</option>
      <option>Dolly</option>
      <option>Tracking</option>
      <option>Zoom</option>
      <option>Handheld</option>
    </select>

    <label htmlFor="shot-notes">Additional Notes</label>
    <textarea
      id="shot-notes"
      name="notes"
      value={form.notes}
      onChange={updateField}
      placeholder="Add any additional notes"
    />

    {error && <p className="error-text">{error}</p>}

    <button className="primary-button" type="submit">
      Save Shot
    </button>
    <button
      className="secondary-button"
      type="button"
      onClick={() => setShowForm(false)}
    >
      Cancel
    </button>
  </form>
)}

{shots.length === 0 ? (
  <StatusMessage
    status="empty"
    message="No shots yet. Add your first shot to begin planning."
  />
) : view === "table" ? (
```

```
<div className="table-wrapper">
  <table className="shot-table">
    <thead>
      <tr>
        <th>Number</th>
        <th>Shot</th>
        <th>Size</th>
        <th>Angle</th>
        <th>Movement</th>
        <th>Notes</th>
        <th>Actions</th>
      </tr>
    </thead>

    <tbody>
      {shots.map((shot, index) => (
        <tr key={shot.id}>
          <td>{shot.number}</td>
          <td>{shot.title}</td>
          <td>{shot.size || "—"}</td>
          <td>{shot.angle || "—"}</td>
          <td>{shot.movement || "—"}</td>
          <td>{shot.notes || "—"}</td>
          <td>
            <button
              className="secondary-button"
              onClick={() => onEditShot?.(shot)}
            >
              Edit
            </button>
            <button
              className="danger-button"
              onClick={() => deleteShot(shot.id)}
            >
              Delete
            </button>
            <button
              className="secondary-button"
              onClick={() => moveShot(index, -1)}
              disabled={index === 0}
            >
              ↑
            </button>
            <button
              className="secondary-button"
              onClick={() => moveShot(index, 1)}
              disabled={index === shots.length - 1}
            >
```

```
                ↓
              </button>
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
) : (
  <div className="project-grid">
    {shots.map((shot, index) => (
      <article className="frame-card shot-card" key={shot.id}>
        <span className="project-label">Shot {shot.number}</span>
        <h3>{shot.title}</h3>
        <p>{shot.description}</p>
        <p><strong>Size:</strong> {shot.size || "Not selected"}</p>
        <p><strong>Angle:</strong> {shot.angle || "Not selected"}</p>
        <p><strong>Movement:</strong> {shot.movement || "Not selected"}</p>
        <p><strong>Notes:</strong> {shot.notes || "None"}</p>

        <div className="card-actions">
          <button
            className="secondary-button"
            onClick={() => onEditShot?.(shot)}
          >
            Edit
          </button>
          <button
            className="danger-button"
            onClick={() => deleteShot(shot.id)}
          >
            Delete
          </button>
          <button
            className="secondary-button"
            onClick={() => moveShot(index, -1)}
            disabled={index === 0}
          >
            Move Up
          </button>
          <button
            className="secondary-button"
            onClick={() => moveShot(index, 1)}
            disabled={index === shots.length - 1}
          >
            Move Down
          </button>
        </div>
```

```
            </article>
          ))}
        </div>
      )}
    </section>
  );
}
```